WSABI Summer Research Lab **Summer 2026**

Lab 16: Tree-Based Models

Instructor: Jonathan Pipping-Gamón *Authors: JP, RB*

16.1 Estimating NFL Win Probability

Use `16_win-probability.csv` to estimate NFL win probability from play-by-play data. Each row is one non-kneel, non-overtime play from a regular-season game. Tied games are omitted, so each row has a winner and loser. The response is `posteam_win`, which equals 1 if the team with possession before the play eventually won the game and 0 if it eventually lost.

Let x_i denote the game state before play i. The pre-play fields available for modeling include:

- field position: `yardline_100`, the yards from the opponent's goal line;
- down and distance: `down` and `ydstogo`;
- score context: `score_differential`, `posteam_score`, and `defteam_score`;
- clock context: `qtr`, `half_seconds_remaining`, and `game_seconds_remaining`;
- timeout context: `posteam_timeouts_remaining` and `defteam_timeouts_remaining`;
- team context: `posteam_spread` and `posteam_type`.

The file also includes `winner_team` and `loser_team` to document the game result. Do not use those outcome fields as predictors. The target is the conditional probability

$$\mathrm{WP}(x) = \mathbb{P}(\texttt{posteam\_win} = 1 \mid X = x).$$

16.2 Train, Validation, and Test Seasons

The goal is to predict future games, so use a season-based split:

- train on 2021–2023 plays;
- validate on 2024 plays;
- test on 2025 plays.

Tune models using only the training and validation seasons. After the tuning decisions are fixed, refit each selected model on 2021–2024 plays and report its log loss on the 2025 test season. Do not use the test season to choose tuning parameters or change model specifications.

16.2.1 Task 1: Tune Four Win-Probability Models

Fit and tune each of the following model classes. For every model class, report the candidate tuning grid, the validation log loss for each candidate or grid point, the selected tuning parameters, and the final test log loss.

1. **Logistic model.** Fit exactly one logistic-regression model. The response is `posteam_win`. Use this formula:

   $$\texttt{posteam\_win} \sim \texttt{score\_differential} * \texttt{game\_seconds\_remaining} + \texttt{yardline\_100} * \texttt{down} + \texttt{ydstogo} + \texttt{qtr} + \texttt{posteam\_timeouts\_remaining} + \texttt{defteam\_timeouts\_remaining} + \texttt{posteam\_spread} + \texttt{posteam\_type}.$$

   Fit the model with `glm(..., family = binomial())`. Do not compare multiple logistic formulas.

2. **Single classification tree.** Fit a classification tree with `rpart`. Tune tree complexity using validation log loss. Your grid should include maximum depth and at least one node-size or pruning parameter.

3. **Random forest.** Fit a probability forest with `ranger`. Tune at least `mtry` and minimum node size. You may keep the number of trees fixed while tuning, then increase it for the final fit.

4. **XGBoost.** Fit boosted trees with `xgboost` and a binary objective. Tune at least learning rate, maximum tree depth, minimum child weight, row subsampling, and column subsampling. Use validation log loss and early stopping to choose the number of boosting rounds within each grid point.

16.2.2 Task 2: Compare Test Performance

Create a table with one row per model class:

Model	Selected tuning parameters	Validation log loss	Test log loss
Logistic model			
Classification tree			
Random forest			
XGBoost			

Then make a bar plot of the final test log loss values. Which model predicts 2025 game winners best? Is the most flexible model also the best model? Explain using the validation and test results.

16.2.3 Task 3: Block Bootstrap Uncertainty

Take the model class with the best final test log loss from Task 2. Before making any plots, create the plotting grids listed in Task 4. Then use the selected tuning parameters for the best model class to quantify uncertainty with a block bootstrap:

1. Let the training data for the final model be the combined 2021–2024 data.
2. For $b = 1, \ldots, 100$, sample games with replacement from the 2021–2024 games.
3. Fit the selected model class to the bootstrapped games. Keep the tuning parameters fixed; do not re-tune inside the bootstrap.

4. Predict win probabilities on each plotting grid from Task 4.

Use the middle 95% of the bootstrap predictions to form pointwise confidence intervals for the plotted win probabilities.

16.2.4 Task 4: Visualize the Model with Uncertainty

Use the selected version of each model class to visualize fitted win probability, and use the bootstrap results from Task 3 to add uncertainty for the best-performing model. Unless the plot varies the displayed quantity, use midfield, first down with `ydstogo = 10`, both teams with 3 timeouts, possession-team spread 0, and `posteam_type = home`. Make the following plots once, with uncertainty included where it applies:

1. A line plot of predicted win probability versus score differential, coloring by quarter and faceting by model class. Add 95% bootstrap ribbons for the best-performing model.
2. A heatmap of predicted win probability as a function of score differential and game seconds remaining for the best-performing model. Include uncertainty by adding a matching heatmap, contour layer, or facet for 95% interval width.
3. A line plot of predicted win probability versus field position for score differentials -7, 0, and 7. Facet by model class and down, and add 95% bootstrap ribbons for the best-performing model.

In your writeup, compare the shapes learned by the four model classes. Identify at least one feature interaction that the tree-based models capture differently from logistic regression. Also explain where the best model's win-probability estimates are most uncertain and whether that pattern matches your intuition about which football situations are common or rare.

References

[Breiman1996] Breiman, L., *Bagging Predictors*, Machine Learning, Vol. 24, pp. 123–140, 1996.

[Breiman2001] Breiman, L., *Random Forests*, Machine Learning, Vol. 45, pp. 5–32, 2001.

[Friedman2001] Friedman, J. H., *Greedy Function Approximation: A Gradient Boosting Machine*, The Annals of Statistics, Vol. 29, No. 5, pp. 1189–1232, 2001.

[ChenGuestrin2016] Chen, T., & Guestrin, C., *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.